Mapping System Components

Week 4 • Day 3 • Technical Architecture & Constraints

TPM Academy

🎯 Day 3 Objectives

- Use **C4 Context** to show people, the system, and external neighbors
- Use **C4 Container** to show deployable units and how they talk
- Always include a **legend** — non-architect-readable
- Mark **trust boundaries** on the Container diagram
- Stress-test the diagrams with another triad and with AI

🕐 Today's Shape

Clock	Block	Outcome
09:00 – 09:15	Opening	Recap §§1–3; whiteboards / paper out
09:15 – 10:30	Block 1 + Activity 1	Context diagram
10:45 – 12:00	Block 2 + Activity 2	Container diagram
13:00 – 14:15	Block 3 + Activity 3	Stress-test the diagram
14:30 – 16:00	Block 4 + Activity 4 + Wrap	AI critique + final polish

Block 1 — The C4 Model

Why Context + Container is TPM scope

📚 The Four C4 Levels

Level	What it shows	Audience	Draw it?
Context	System + users + neighbors	Everyone	✅ Today
Container	Deployable units inside the system	TPMs, architects, devs	✅ Today
Component	Code-level building blocks	Engineers	❌
Code	Class-level detail	Engineers	❌

Discipline: stay above Component. The architect / engineer owns those layers.

🌐 Context Diagram

Answers: **who interacts with this system, and what other systems does it talk to?**

- **People:** direct users + indirect roles (Ops VP, compliance, customer success)
- **The system in scope** — one big box (we explode it next)
- **External systems** — pull from your integration table
- Arrows with **verb-based labels** ("submits reconcile", not just "POST")

> **NOT in Context:** database choices, internal services, framework choices. Cap at 12 boxes.

📋 The Legend (Don't Skip)

```
Legend
- □ Box with no fill   = system / module owned by us
- ■ Box with fill      = external system (out of our control)
- 👤 Stick figure       = person (role)
- → Solid arrow        = synchronous call
- ⇢ Dashed arrow       = async / event
- (label on arrow)     = intent in plain English
```

> **A diagram without a legend is a Rorschach test.** Force the legend.

👥 Activity 1 — Context Diagram

Triads • 35 minutes

List people; list externals; draw it; legend; 60-second test with another triad.

- 5 min: list people (incl. indirect)
- 5 min: list externals (pull from integration table)
- 15 min: draw on whiteboard / paper
- 5 min: add the legend
- 5 min: 60-sec walk-through with another triad

⏱️ 5 + 5 + 15 + 5 + 5

Block 2 — Container Diagram

Inside the system: what deploys, how they talk

📥 What's a "Container" in C4?

A container is **anything you would update independently**: a web app, a mobile app, a service, a database, a queue, a CDN, an event bus.

- Mobile app (iOS + Android = two containers)
- Backend monolith (one container, even if internally modular)
- Each datastore is its own container
- Each queue / event bus is its own container

Modular monolith stance: draw modules *inside* the monolith container as labeled sub-boxes. Don't promote them.

📝 Worked Container Diagram (FieldPulse)

👥 Activity 2 — Container Diagram

Triads • 40 minutes

List containers; draw the diagram; annotate modules inside the monolith container; 90-second test.

- 10 min: list containers + tech (rough) + owners
- 15 min: draw containers + protocol-labeled arrows
- 10 min: annotate modular split inside containers
- 5 min: 90-sec walk-through

⏱️ 10 + 15 + 10 + 5

Block 3 — Stress-Test the Diagram

Three lenses: failure, trust, evolvability

🔎 The Three Lenses

Lens	Question
Failure	Trace a failure: "What happens when the audit topic is full?" Walk the diagram.
Trust boundary	Where does data cross from our control to someone else's? Mark with a thick line.
Evolvability	If the architecture stance changes (modular monolith → microservice), which arrows change?

> **Trust boundary:** the most-missed annotation. It's where compliance and security risks concentrate.

👥 Activity 3 — Stress-Test

Triad pairs • 40 minutes

Swap diagrams. Failure trace. Trust boundary mark-up. Evolvability question.

- 5 min: swap + read
- 10 min: failure trace (one scenario aloud)
- 10 min: trust boundary mark-up
- 10 min: evolvability question
- 5 min: authors annotate diagram with findings

⏱️ 5 + 10 + 10 + 10 + 5

Block 4 — AI Critique + Final Polish

Architect's eye + stakeholder's eye

Today's Two Prompts

A. Architect critique

"What's wrong with this diagram or the stance it implies?" 3 issues, each with scenario.

B. Stakeholder Q's

"Operations VP — what 5 questions would you ask after seeing this?" Non-engineer questions.

```
Role: Principal architect reviewing a feature's C4 Container diagram.
Context: <plain-text description:="" containers="" +="" tech="" arrows="" owners="">
Stance: <copy from="" tcd="" §1="">
Task: Identify the top 3 issues. For each, name the specific scenario.
Constraints:
- Treat ownership and tech as given, not as targets
- Flag any unstated assumption
- Do not suggest generic 'modernization'
Format: Numbered list — Issue / Scenario / Suggested clarification.</copy></plain-text>
```

👥 Activity 4 — AI Critique + Polish

Triads • 45 minutes • Wrap

Run both prompts. Decide what to address. Polish diagrams. Provenance note.

- 10 min: Prompt A → 3 issues
- 10 min: adopt / defer / reject
- 10 min: Prompt B → stakeholder questions
- 10 min: final polish (legend, trust boundary, key arrows)
- 5 min: provenance note

⏱️ 10 + 10 + 10 + 10 + 5 · 15-min wrap

🎉 Day 3 Wrap

What we built

- C4 Context diagram
- C4 Container diagram
- Trust-boundary annotation
- Stress-tested with peers + AI
- TCD §2 diagram-backed

What opens tomorrow

- **Latency, availability, rate limits**
- SLOs + error budgets
- Latency budgets per hop
- Rate limits as design tool

Bring to Day 4: Your two diagrams. The Container diagram is the input to latency-budgeting tomorrow.

💬 Questions & Reflection

What does your diagram make obvious that words couldn't?